Types of Errors

Contents

- LRC – Linear Block Codes
- CRC – Cyclic Redundancy Check
- CHECKSUM

Linear Block Codes

- Almost all block codes used today belong to a subset called linear block codes.
- A linear block code is a code in which the XOR (addition modulo-2) of two valid codewords creates another valid codeword.

Contd…

In a linear block code, the exclusive OR (XOR) of any two valid codewords creates another valid codeword.

Contd...

In a linear block code, the minimum Hamming distance is the number of 1s in the nonzero valid codeword with the smallest number of 1s.

10010101

In our first code (Table 10.1), the numbers of 1s in the nonzero codewords are 2, 2, and 2. So the minimum Hamming distance is $d_{min} = 2$.

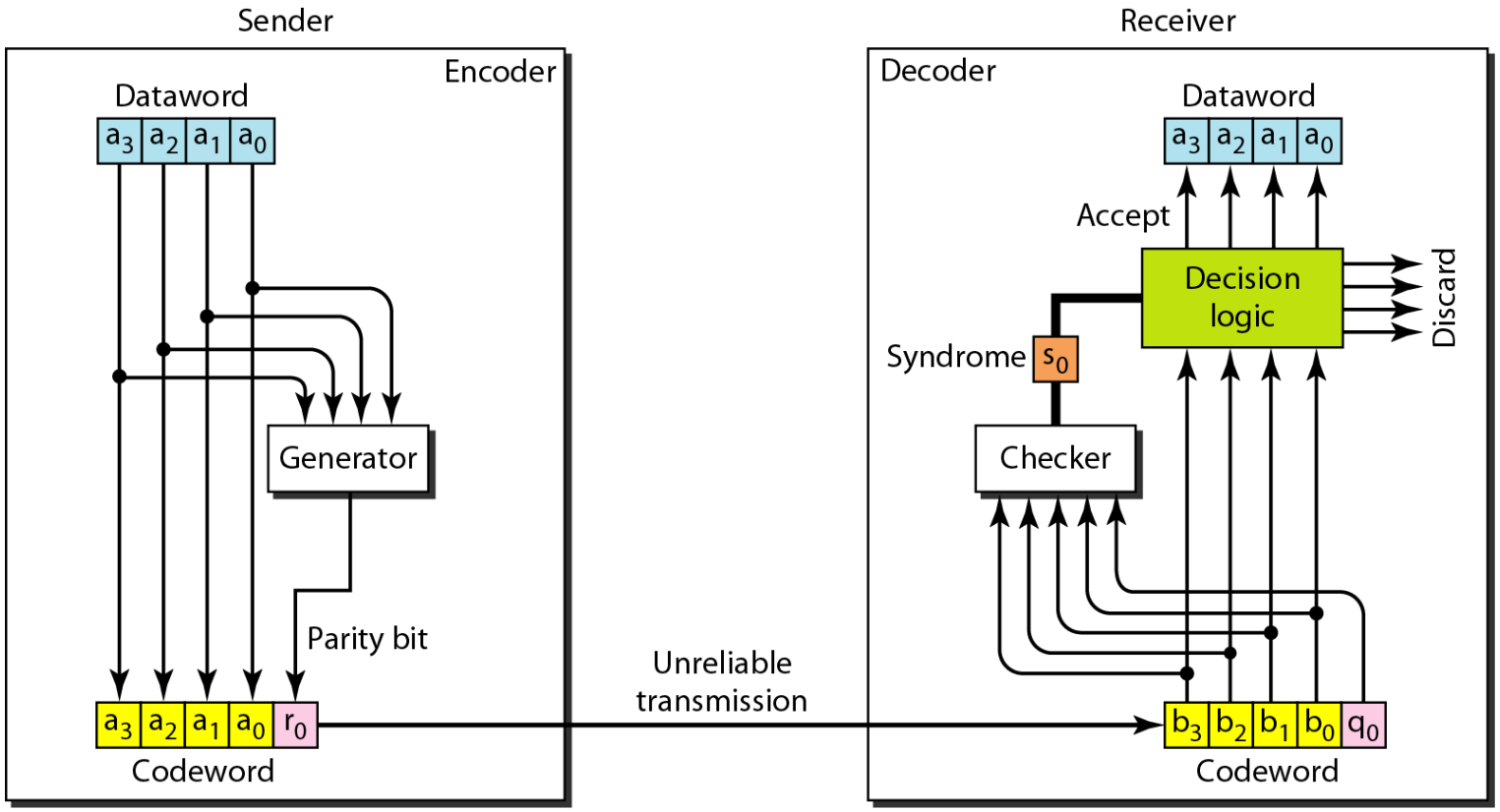
Contd...

A simple parity-check code is a single-bit error-detecting code in which $n = k + 1$ with $d_{min} = 2$.

Simple parity-check code

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	**00000**	1000	**10001**
0001	**00011**	1001	**10010**
0010	**00101**	1010	**10100**
0011	**00110**	1011	**10111**
0100	**01001**	1100	**11000**
0101	**01010**	1101	**11011**
0110	**01100**	1110	**11101**
0111	**01111**	1111	**11110**

Encoder and decoder for simple parity-check code



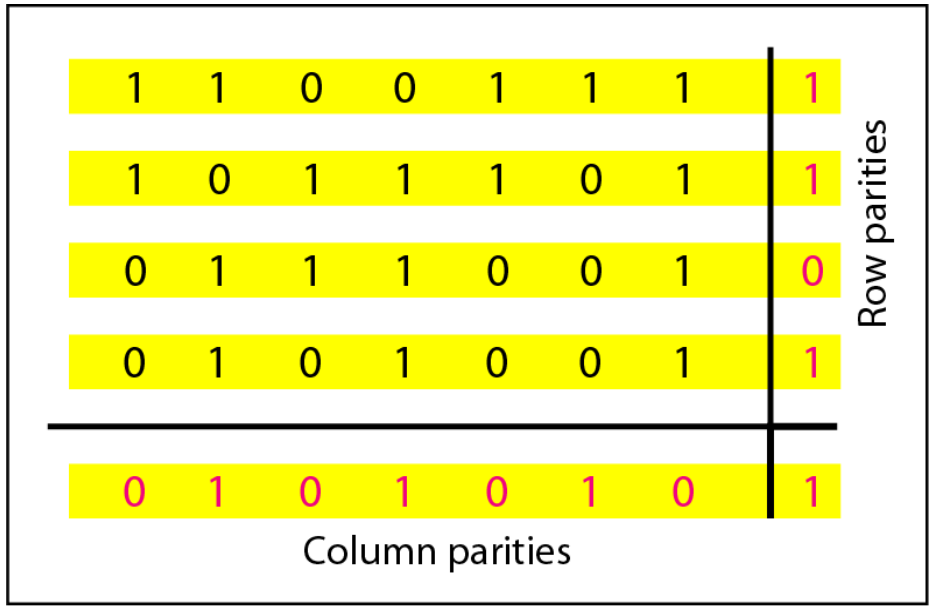
Parity-check code

- $r_0 = a_3 + a_2 + a_1 + a_0$ (modulo-2)
- Syndrome (calculated by the receiver)
- $s_0 = b_3 + b_2 + b_1 + b_0 + q_0$ (modulo-2)

Contd...

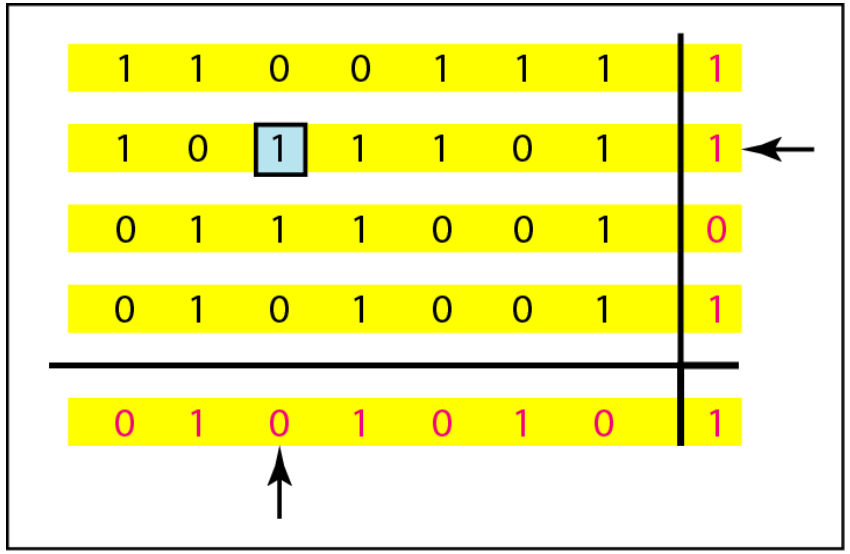
A simple parity-check code can detect an odd number of errors.

Two-dimensional parity-check code

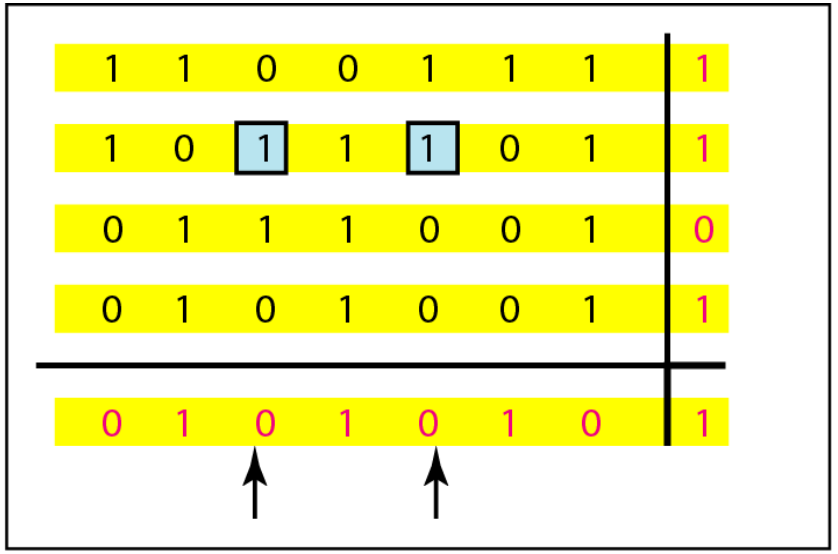


a. Design of row and column parities

Contd…

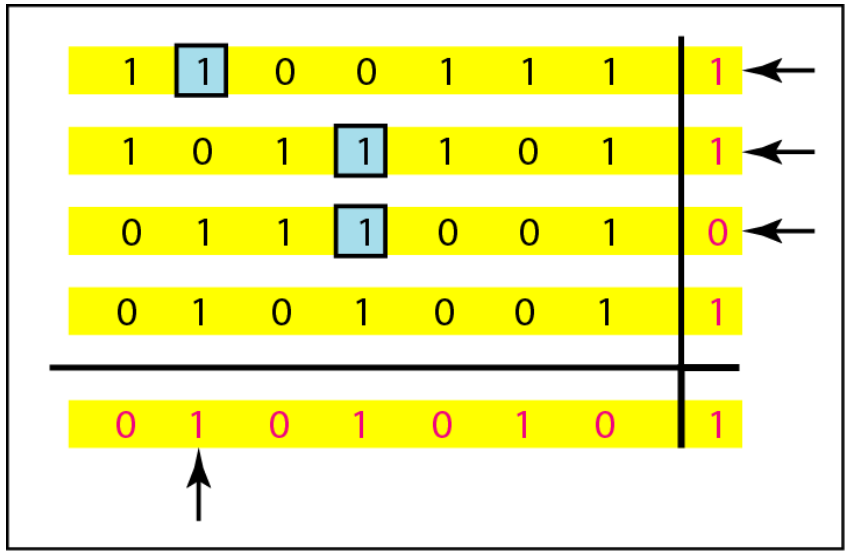


b. One error affects two parities

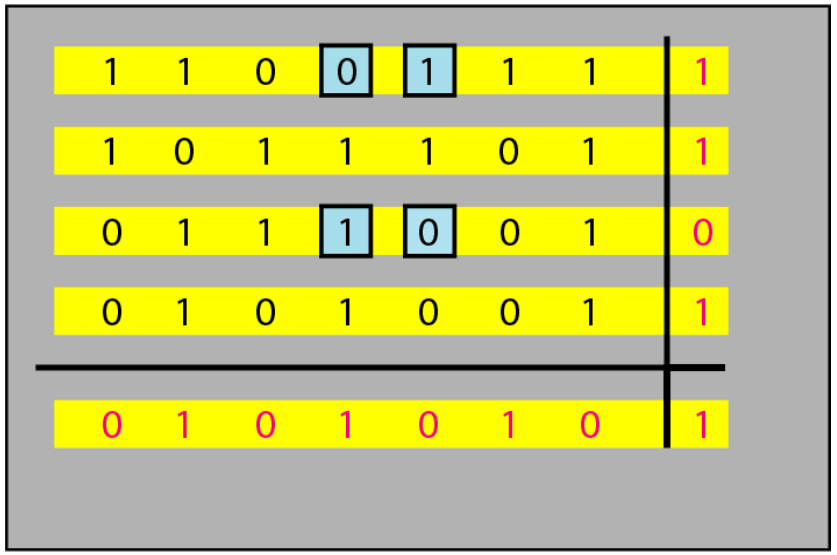


c. Two errors affect two parities

Contd…



d. Three errors affect four parities



e. Four errors cannot be detected

Two-dimensional parity-check can detect up to 3-bit errors

Hamming Code

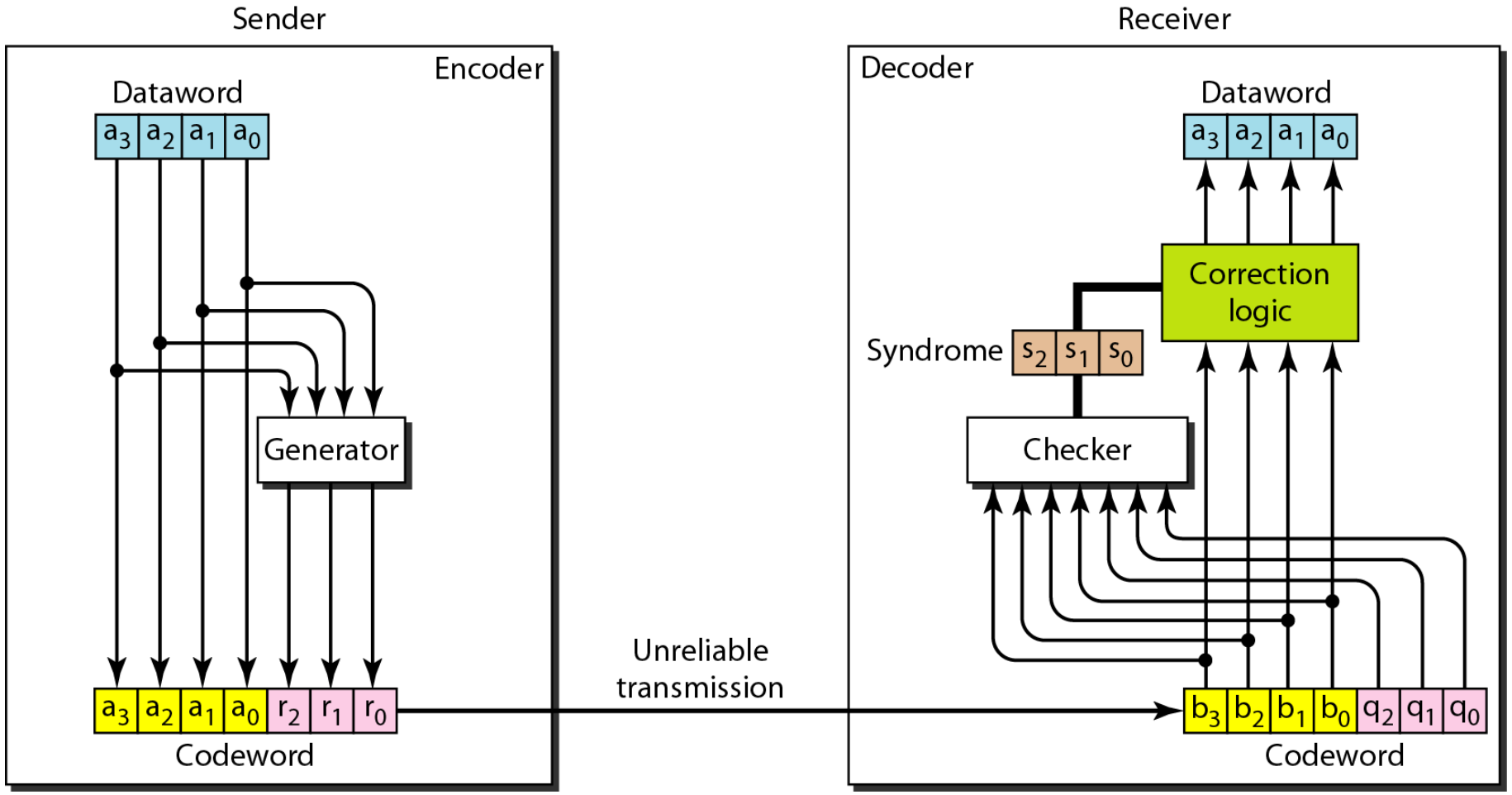
Hamming Code

- Parity checks are created as follow (using modulo-2)
  - $r_0 = a_2 + a_1 + a_0$
  - $r_1 = a_3 + a_2 + a_1$
  - $r_2 = a_1 + a_0 + a_3$

Hamming code C(7, 4) ***C(n,k)*** $d_{min}=3$

Datawords	*Codewords*	*Datawords*	*Codewords*
0000	0000000	1000	1000110
0001	0001101	1001	1001011
0010	0010111	1010	1010001
0011	0011010	1011	1011100
0100	0100011	1100	1100101
0101	0101110	1101	1101000
0110	0110100	1110	1110010
0111	0111001	1111	1111111

The structure of the encoder and decoder for a Hamming code



Contd...

- The checker in the decoder creates a 3-bit syndrome ($s_2s_1s_0$).
- In which each bit is the parity check for 4 out of the 7 bits in the received codeword:
  - $s_0 = b_2 + b_1 + b_0 + q_0$
  - $s_1 = b_3 + b_2 + b_1 + q_1$
  - $s_2 = b_1 + b_0 + b_3 + q_2$
- The equations used by the checker are the same as those used by the generator with the parity-check bits added to the right-hand side of the equation.

Logical decision made by the correction logic analyzer

Syndrome	000	001	010	011	100	101	110	111
Error	None	q_0	q_1	b_2	q_2	b_0	b_3	b_1

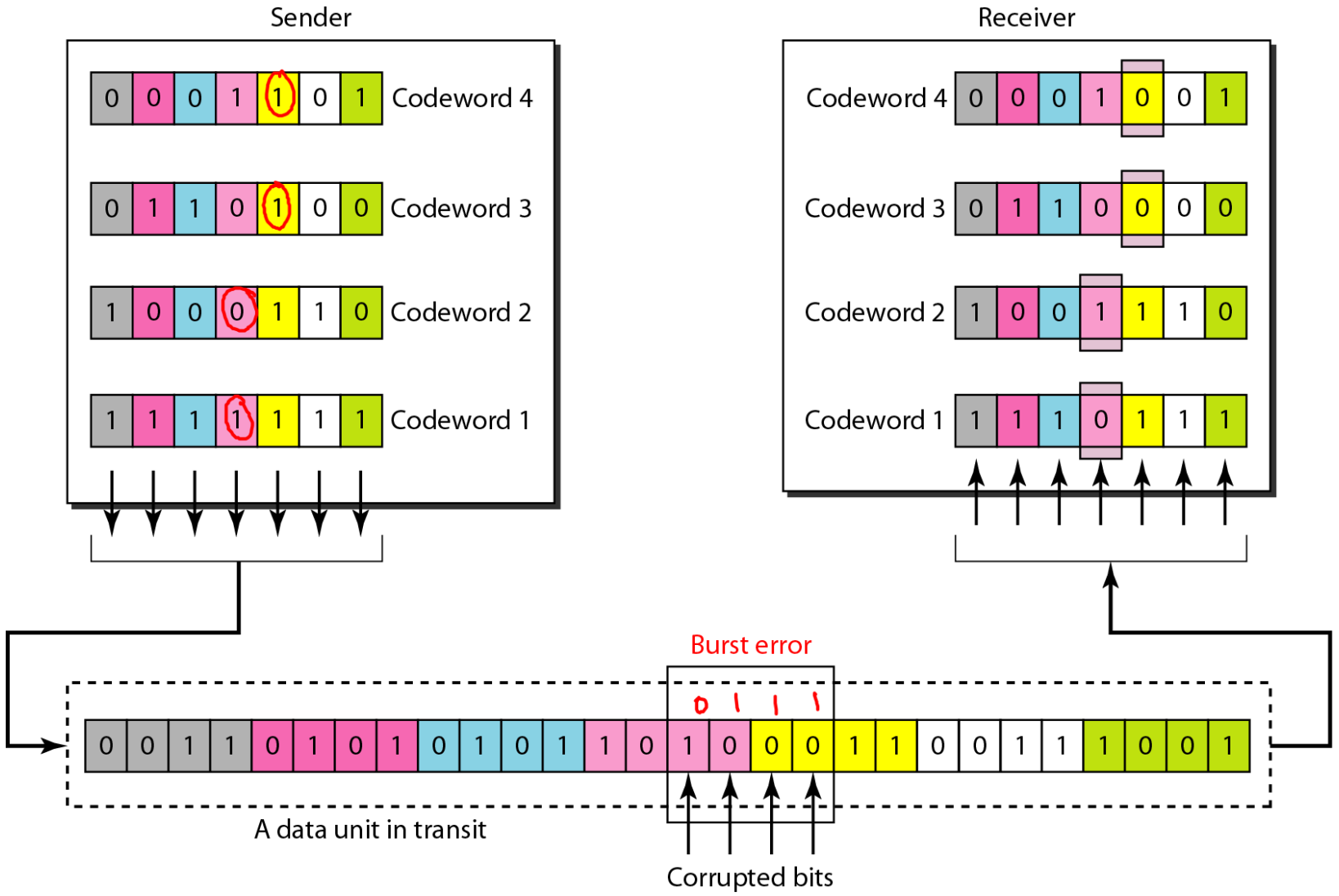
Hamming code C(7, 4) can :

- detect up to 2-bit error (d_{min} -1)
- can correct up to 1 bit error (d_{min}-1)/2

Activity – Concept Check

- Given the data word of 0010 for a (7,4) hamming code, show the generation of the codeword using even parity. assume that at the receiver, the q_0 bit in the codeword is received with an error. with suitable illustration, explain the error detection and correction process using the decoder.

Burst error correction using Hamming code



Split burst error between multiple codewords

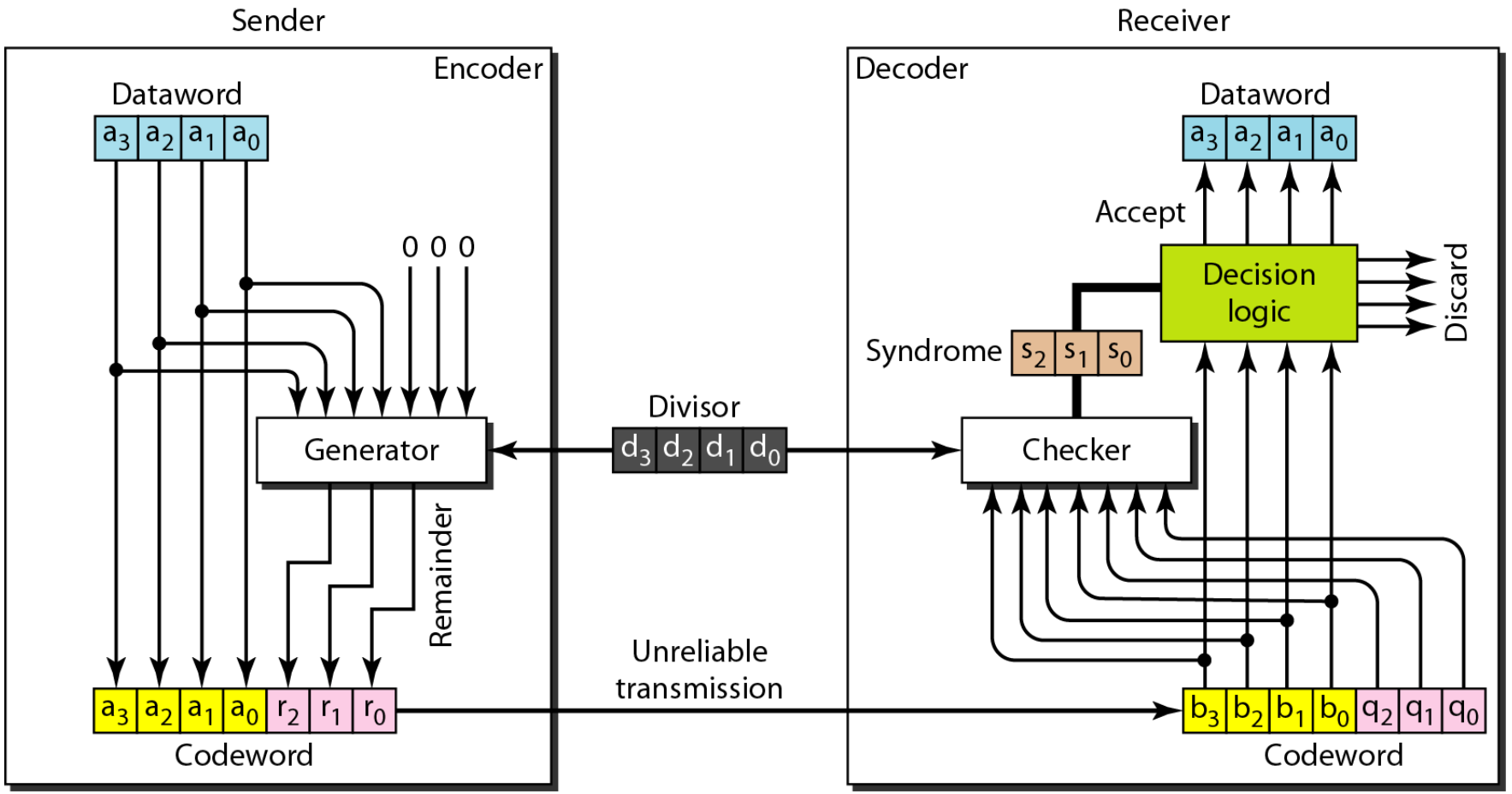
CYCLIC CODES

- Cyclic codes are special linear block codes with one extra property.
- In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.

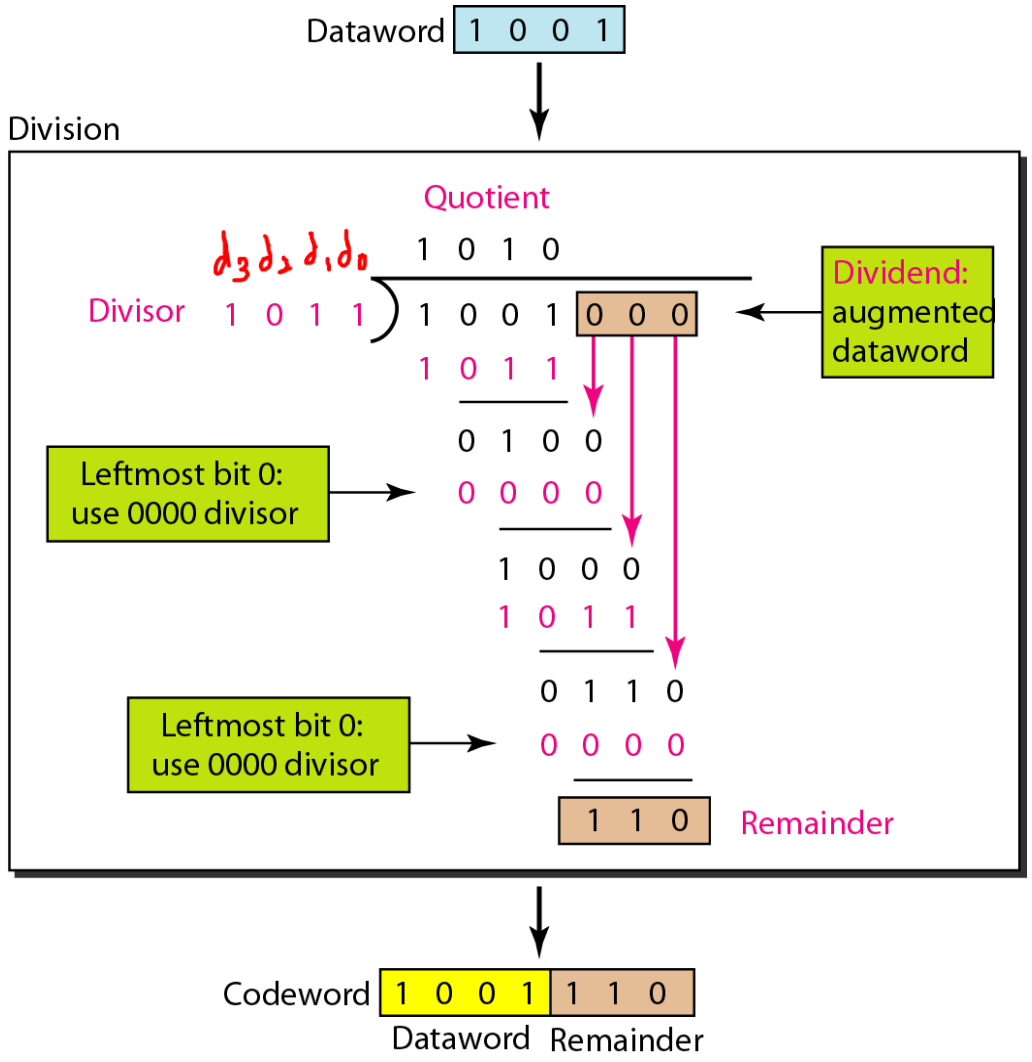
A CRC code with C(7, 4)

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

CRC encoder and decoder

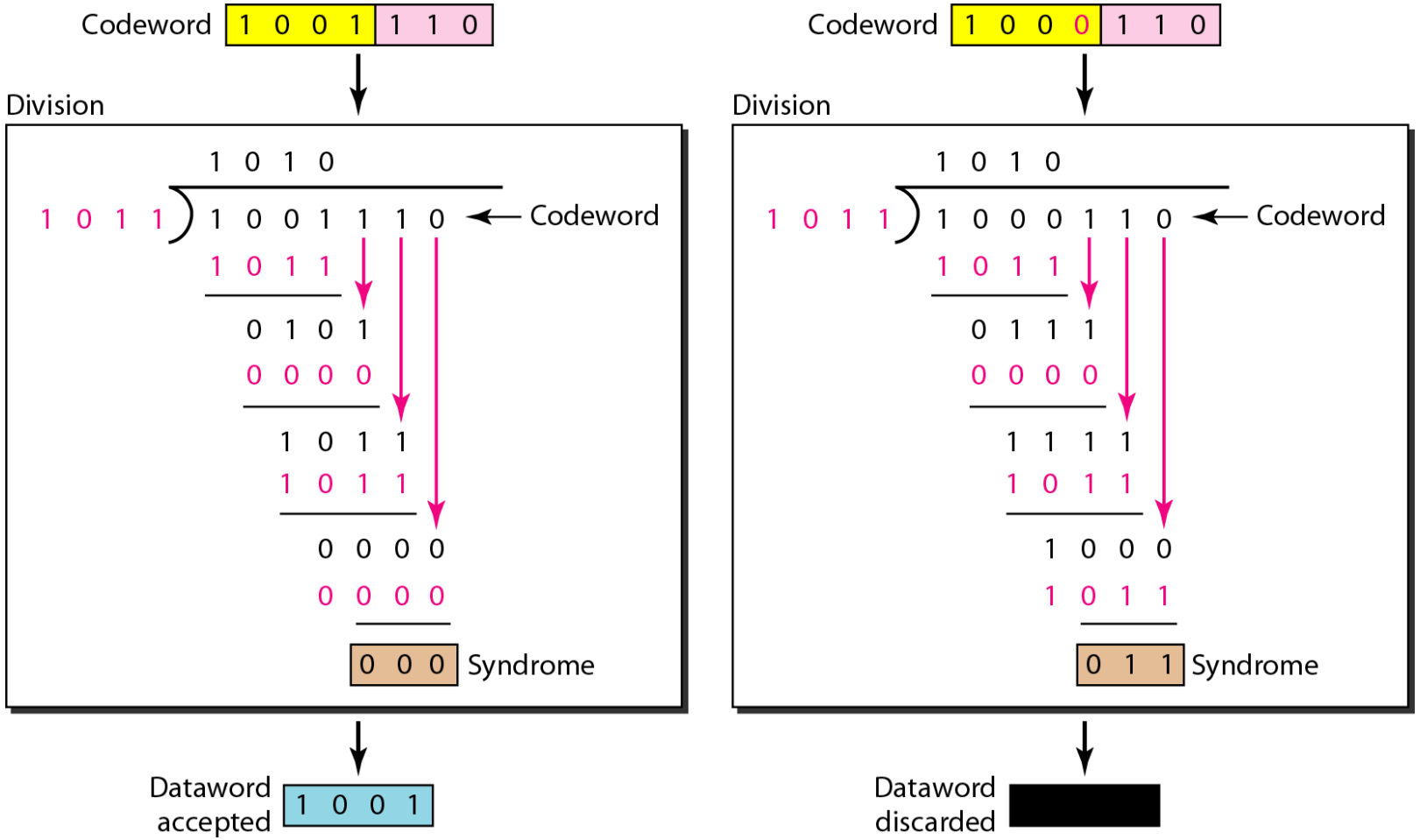


Division in CRC encoder



00 0
01 1
10 1
11 0

Division in the CRC decoder for two cases



CRC Polynomials

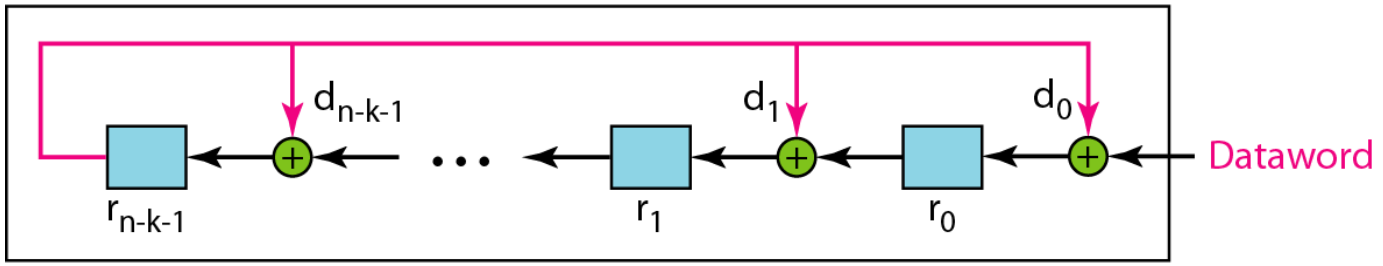
General design of encoder and decoder of a CRC code

Note:

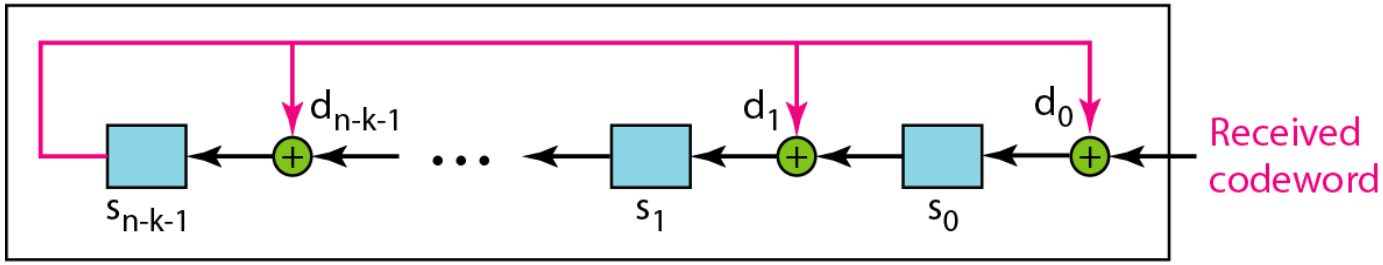
The divisor line and XOR are missing if the corresponding bit in the divisor is 0.

$x^3 + x^2 + x + 1 = 1101$

$x^4 + x^2 + x = 10110$



a. Encoder



b. Decoder

CRC - Polynomials

- A pattern of 0s and 1s can be represented as a polynomial with coefficients of 0 and 1.
- The power of each term shows the position of the bit; the coefficient shows the value of the bit.
- Degree of a Polynomial: The degree of a polynomial is the highest power in the polynomial. For example, the degree of the polynomial $x^6 + x + 1$ is 6. The degree of a polynomial is 1 less than the number of bits in the pattern. The bit pattern in this case has 7 bits.

Contd...

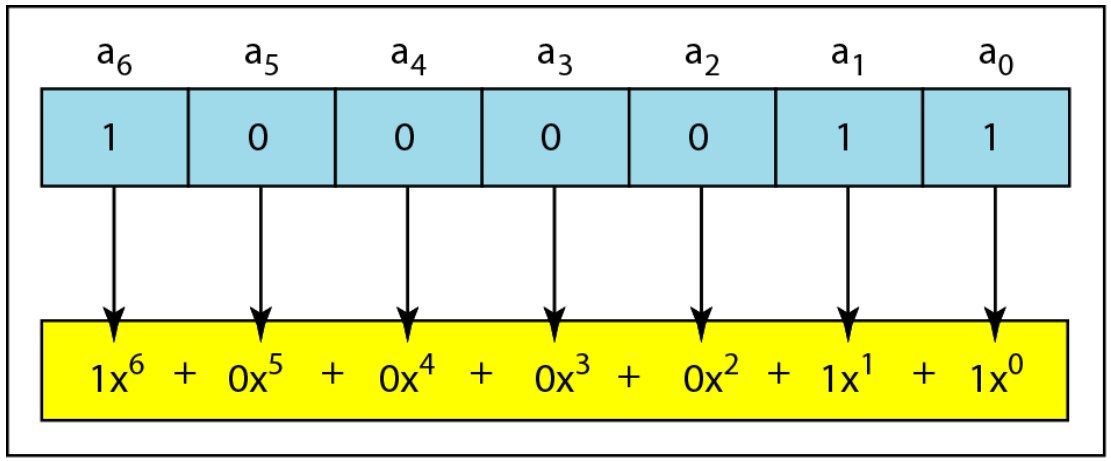
- Adding and Subtracting Polynomials: Adding and subtracting polynomials in mathematics are done by adding or subtracting the coefficients of terms with the same power.
- For example, adding $x^5 + x^4 + x^2$ and $x^6 + x^4 + x^2$ gives just $x^6 + x^5$. The terms x^4 and x^2 are deleted

Contd...

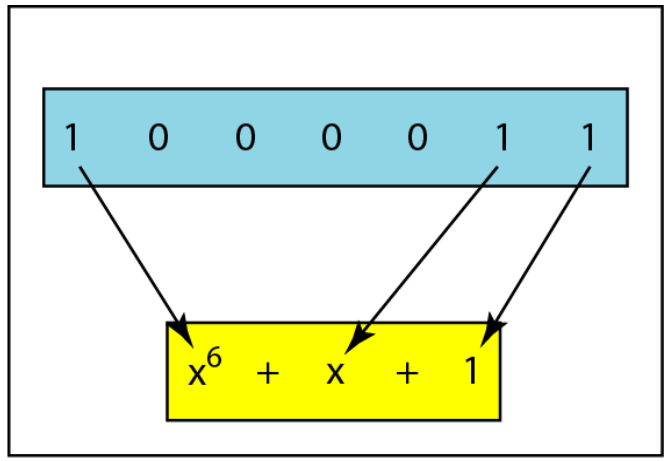
- Multiplying Two Polynomials : Multiplying a polynomial by another is done term by term. Each term of the first polynomial must be multiplied by all terms of the second.
- The result is pairs of equal terms are deleted

$$(x^5+x^3+x^2+x)(x^2+x+1) = x^7+x^6+x^5+x^5+x^4+x^3+x^4+x^3+x^2+x^3+x^2+x$$
$$= x^7+x^6+x^3+x$$

Polynomial to represent a binary word

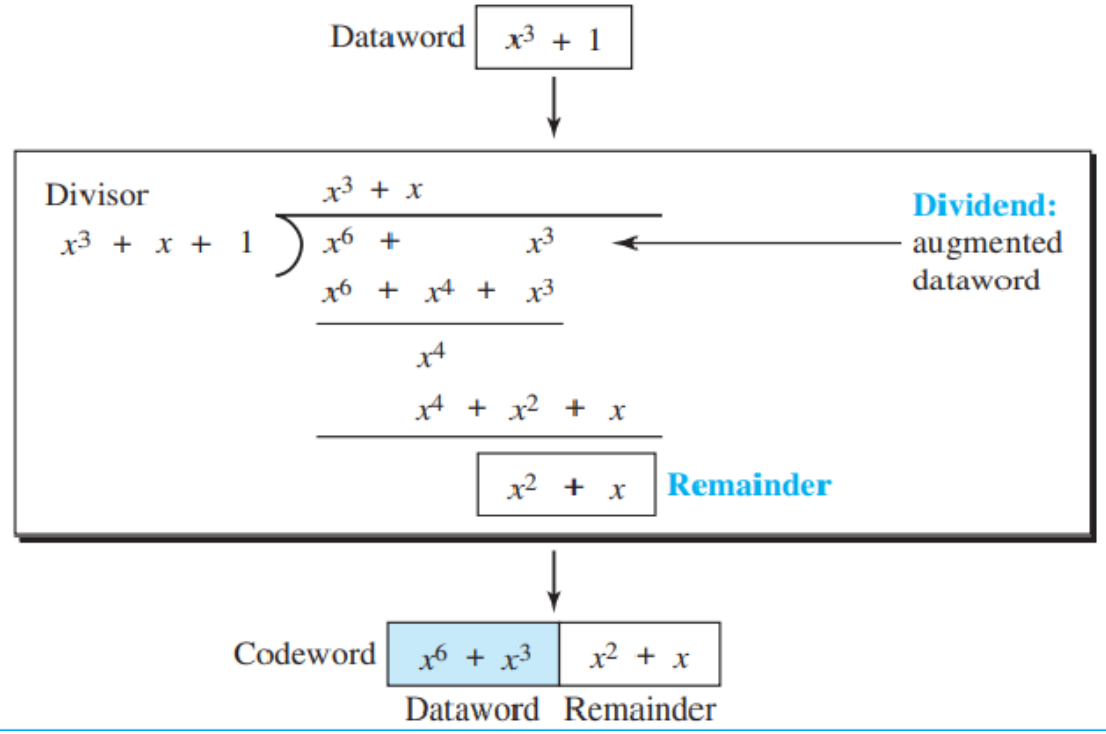


a. Binary pattern and polynomial



b. Short form

Cyclic Code Encoder Using Polynomials



CRC (Cyclic Redundancy Check) – Example (Polynomial Division)

Dataword $D(x) = x^3 + 1$ (Binary: 1001) **Generator Polynomial** $G(x) = x^3 + x + 1$ (Binary: 1011) Degree of $G(x) = 3$

TRANSMITTER SIDE – CRC GENERATION

1 Dataword

$D(x) = x^3 + 1$
(Binary: 1001)

2 Append 3 zeros (Degree of $G(x) = 3$)

Multiply $D(x)$ by x^3
$(x^3 + 1)\ x^3 = x^6 + x^3$
(Binary: 1001000)

Dividend (Augmented Dataword)
$x^6 + x^3$

3 Divide the dividend by $G(x)$ using modulo-2 division (XOR)

Divisor
$G(x) = x^3 + x + 1$
(Binary: 1011)

$$
\begin{array}{r|l}
 & \text{Quotient} = x^3 + x \\
x^3 + x + 1 & x^6 + x^3 \\
 & x^6 + x^4 + x^3 \quad \text{(XOR)} \\
\hline
 & x^4 \\
 & x^4 + x^2 + x \quad \text{(XOR)} \\
\hline
 & x^2 + x \quad \leftarrow \text{Remainder}
\end{array}
$$

4 Remainder (CRC)

$R(x) = x^2 + x$
(Binary: 110)

5 Form the Codeword

Codeword $C(x) = x^6 + x^3 + x^2 + x$
(Binary: 1001110)

Dataword 1001 + CRC (Remainder) 110 → Codeword (Transmitted) 1001110

The transmitter sends the codeword 1001110 (which is Dataword + CRC) to the receiver.

RECEIVER SIDE – CRC CHECKING

1 Received Codeword

$C(x) = x^6 + x^3 + x^2 + x$
(Binary: 1001110)

2 Divide the received codeword by the same generator polynomial $G(x)$

Divisor
$G(x) = x^3 + x + 1$
(Binary: 1011)

$$
\begin{array}{r|l}
 & \text{Quotient} = x^3 + x \\
x^3 + x + 1 & x^6 + x^3 + x^2 + x \\
 & x^6 + x^4 + x^3 \\
\hline
 & x^4 + x^2 + x \\
 & x^4 + x^2 + x \\
\hline
 & 0
\end{array}
$$

Step (XOR)

1) $x^6 + x^3 + x^2 + x \oplus (x^6 + x^4 + x^3) = x^4 + x^2 + x$
2) $x^4 + x^2 + x \oplus (x^4 + x^2 + x) = 0$

3 Check Remainder

Remainder = 0

Remainder is 0 ⇒ No error detected.
The received data is accepted.

If remainder ≠ 0
Error is detected. Data is rejected or retransmision is requested.

SUMMARY

- Generator Polynomial: $G(x) = x^3 + x + 1$ (1011)
- Dataword: $x^3 + 1$ (1001)
- CRC (Remainder): $x^2 + x$ (110)
- Codeword Transmitted: $x^6 + x^3 + x^2 + x$ (1001110)

KEY POINTS

- Number of CRC bits = Degree of Generator Polynomial = 3
- CRC uses modulo-2 (XOR) division
- At receiver, if remainder = 0 ⇒ No error
- If remainder ≠ 0 ⇒ Error detected

XOR Rules

$1 \oplus 1 = 0$
$1 \oplus 0 = 1$
$0 \oplus 1 = 1$
$0 \oplus 0 = 0$

Contd...

The divisor in a cyclic code is normally called the generator polynomial or simply the generator.

Standard polynomials

Name	*Polynomial*	*Application*
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Advantages

- cyclic codes have a very good performance in detecting single-bit errors, double errors, an odd number of errors, and burst errors.
- They can easily be implemented in hardware and software.
- They are especially fast when implemented in hardware.

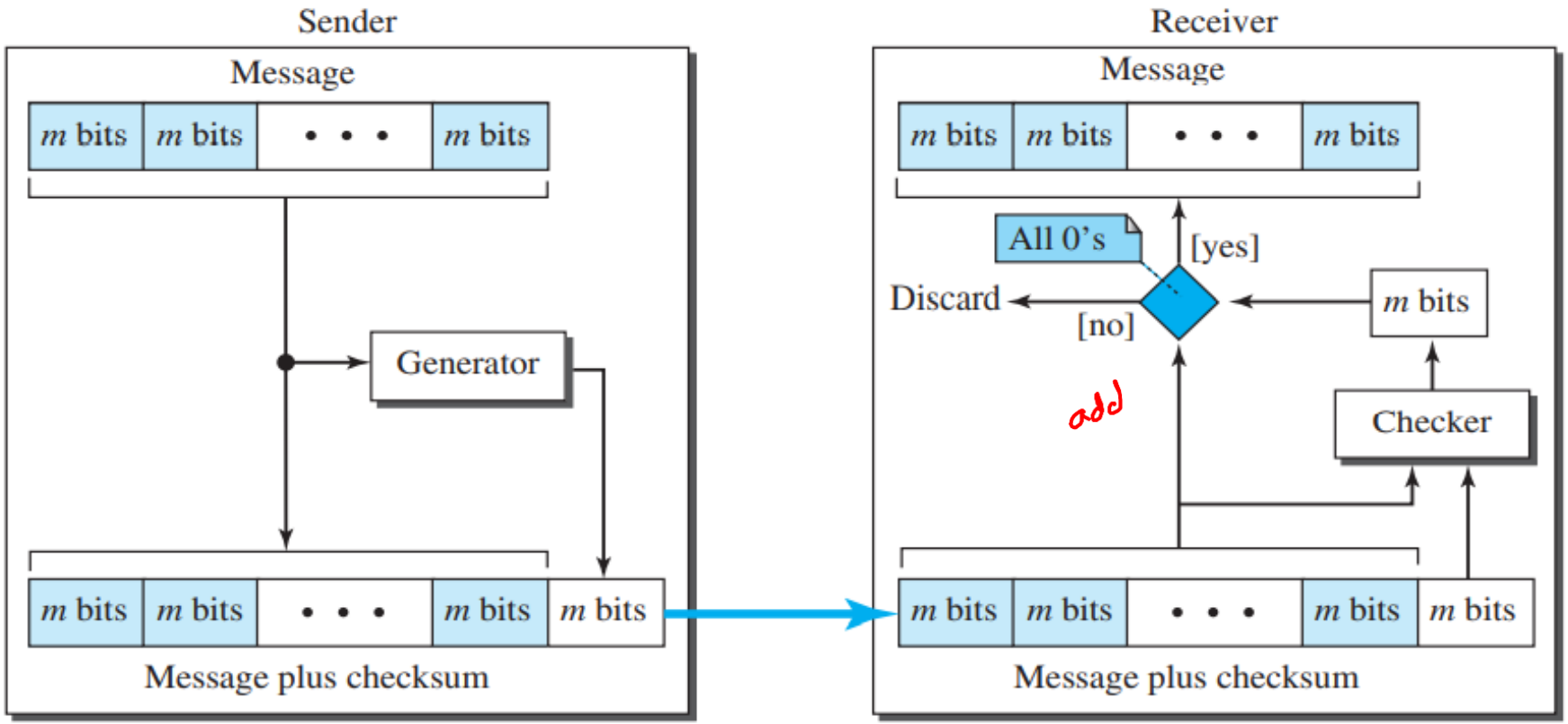
CheckSUM

CHECKSUM

- The last error detection method is called the checksum.
- Checksum is an error-detecting technique that can be applied to a message of any length.
- In the Internet, the checksum technique is mostly used at the network and transport layer rather than the data-link layer

Contd…

- At the source, the message is first divided into m-bit units.
- The generator then creates an extra m-bit unit called the checksum, which is sent with the message.
- At the destination, the checker creates a new checksum from the combination of the message and sent checksum.
- If the new checksum is all 0s, the message is accepted; otherwise, the message is discarded


Sender
Message
m bits
m bits
• • •
m bits
Generator
m bits
m bits
• • •
m bits
m bits
Message plus checksum
Receiver
Message
m bits
m bits
• • •
m bits
All 0's
[yes]
Discard
[no]
m bits
add
Checker
m bits
m bits
• • •
m bits
m bits
Message plus checksum

At the Sender Side

- Divide the data into equal-sized segments (commonly 8-bit, 16-bit, or 32-bit words).
- Add all the segments using **1's complement arithmetic**.
- If there is an overflow (carry), add the carry back to the least significant bit (end-around carry).
- Take the **1's complement** (invert all bits) of the final sum.
- This value is the **checksum**, which is transmitted along with the data.

At the Receiver Side

- Receive both the data and the checksum.
- Add all the data segments and the received checksum using **1's complement addition**.
- If the result is **all 1s (111...111)**, the data is accepted.
- Otherwise, an error is detected, and the frame is discarded or retransmission is requested.

CHECKSUM EXAMPLE (8-BIT)

1's Complement Addition (End-Around Carry)

SENDER SIDE

1 Data to be Sent

Data Block	8-bit Binary	Decimal
D1	10110010	178
D2	01100101	101
D3	01011100	92

2 Add all Data (1's Complement Addition)

```
  10110010   (D1)
+ 01100101   (D2)
--------------------
1 00010111   (Carry 1)
```

Add carry to LSB

```
  00010111
+ 00000001
----------------
  00011000
```

```
  00011000
+ 01011100   (D3)
----------------
  01110100   ← Final Sum
```

3 Compute Checksum (1's Complement of Final Sum)

Final Sum	01110100

↓ 1's Complement

Checksum	10001011

4 Data Transmitted

D1	D2	D3	Checksum
10110010	01100101	01011100	10001011

→ Data is sent over Communication Channel →

RECEIVER SIDE

1 Data Received

Data Block	8-bit Binary
D1	10110010
D2	01100101
D3	01011100
Checksum	10001011

2 Add all Received Data + Checksum (1's Complement Addition)

```
  10110010   (D1)
+ 01100101   (D2)
--------------------
1 00010111   (Carry 1)
```

Add carry to LSB

```
  00010111
+ 00000001
----------------
  00011000
```

```
  00011000
+ 01011100   (D3)
--------------------
  01110100
```

```
  01110100
+ 10001011   (Checksum)
----------------
  11111111   ← Final Sum
```

3 Take 1's Complement of Final Sum

Final Sum
11111111

1's Complement →

Result
00000000

4 Check Result

Result is 00000000 (Zero)

So, **No Error**

Data is Accepted ✓

Key Points

- Sender adds all data bytes using 1's complement addition.
- Checksum = 1's complement of the final sum.
- Receiver adds all received data including checksum.
- If final sum = 11111111 (or complement = 00000000) → No Error.
- Otherwise → Error Detected.

Legend

Data | Checksum | Carry | Final Step